

Progetto Scalable and Cloud Programming: Analisi di Co-occorrenza Sismica con Spark

Sara Casadio

Anno Accademico 2025-26

1 Introduzione

L'obiettivo del progetto è implementare un'applicazione distribuita tramite Apache Spark per l'analisi di dataset sismici. Nello specifico, l'algoritmo ricerca la coppia di località (approssimate alla prima cifra decimale) che presentano il maggior numero di co-occorrenze di eventi sismici nella stessa data, elencando le date in ordine cronologico.

2 Descrizione dell'Approccio

Il problema è stato affrontato utilizzando l'approccio Map-Reduce.

2.1 Pre-processing e Pulizia

I dati sono stati pre-processati come segue:

- Latitudine e longitudine sono state arrotondate alla prima cifra decimale.
- La data è stata troncata per considerare una finestra temporale di 24 ore (YYYY-MM-DD).
- È stata applicata la trasformazione `distinct()` per evitare che più scosse nella stessa zona e nello stesso giorno venissero contate come co-occorrenze multiple.

2.2 Analisi di Co-occorrenza

Per l'analisi delle co-occorrenze è stato implementato un algoritmo basato su un Self-Join del dataset per trovare località diverse che hanno la stessa data. Successivamente per evitare coppie duplicate (A,B) e (B,A), è stato applicato un filtro per mantenere solo le coppie ordinate. Infine, è stato effettuato un conteggio delle occorrenze per ogni coppia di località e sono stati ordinati i risultati in ordine decrescente per ottenere la coppia con il maggior numero di co-occorrenze.

Per ottimizzare l'esecuzione su larga scala, è stato utilizzato il metodo `repartition()` per distribuire uniformemente il carico di lavoro tra i nodi worker.

3 Configurazione Cloud e Deployment

L'analisi è stata eseguita su Google Cloud Platform utilizzando il servizio **Dataproc**. Come richiesto, sono state utilizzate istanze **n2-standard-4** (4 vCPU, 16 GB RAM) con dischi persistenti da 240 GB per garantire prestazioni I/O elevate durante le fasi di *shuffle*.

Il codice è stato compilato in un file JAR tramite `sbt package` e caricato su un bucket Cloud Storage insieme al dataset completo.

4 Analisi di Scalabilità

Per valutare la scalabilità della soluzione, sono stati eseguiti tre test variando il numero di nodi worker. I risultati sono riassunti nella seguente tabella:

N. Worker	Tempo di Esecuzione (min:sec)	Speedup
2	[INSERIRE TEMPO 2W]	1.00
3	[INSERIRE TEMPO 3W]	[CALCOLARE]
4	[INSERIRE TEMPO 4W]	[CALCOLARE]

Tabella 1: Risultati dei test di scalabilità sul dataset completo.

4.1 Discussione dei risultati

[Qui dovrai commentare se il tempo è diminuito linearmente. Ad esempio: "Passando da 2 a 4 worker si è osservato un incremento delle prestazioni del X%, a dimostrazione della corretta parallelizzazione del join."]

5 Conclusioni

L'implementazione ha permesso di individuare con successo la coppia di località con il massimo numero di co-occorrenze. L'utilizzo di Spark su un cluster distribuito ha dimostrato come la gestione oculata delle partizioni sia fondamentale per elaborare dataset di grandi dimensioni in tempi contenuti.